Threat Hunting: Find the Story in the Logs

A beginner notebook for forming a hypothesis, exploring synthetic events, and writing evidence-based findings

NOTEBOOK STYLE Handwriting-inspired typeset notes for printing and revision; not a scan of actual handwriting.

1. What is threat hunting?

Threat hunting is a structured search for suspicious patterns that routine alerts may not have surfaced. A hunter starts with a question, examines available telemetry, compares activity with a normal baseline, and documents what the evidence does and does not show.

THE MINDSET A pattern is a clue, not proof. Keep facts, assumptions, and unknowns separate.

2. A simple hunting loop

1. Ask a focused question, such as "Do several failed logins precede a successful login in this sample?"
2. Choose relevant records and fields: time, user, host, source address, event type, result.
3. Set a clear time window and sort events chronologically.
4. Compare the pattern with the expected baseline and nearby context.
5. Validate the pattern against other benign explanations.
6. Write a short finding with evidence, confidence, limitations, and a next check.

3. Read the sample fields

Field	Plain-language meaning
timestamp_utc	When the training event was recorded (UTC).
host	Synthetic device name, not a real asset.
user	Synthetic account label.
event_type	The kind of activity represented.
source_ip	Reserved documentation IP space used in training data.
result	Success, failure, allowed, or blocked.
label	Teaching label to support discussion; not a real incident verdict.

4. Practice: investigate a login pattern

Open threat-hunt-auth-dns-samples.csv. Every row is fabricated. Look for one account with several failures followed by a success, then check time, host, and source context.

1. Record the case IDs and timestamps that support your hypothesis.
2. Check whether the account and host appear elsewhere in the same sample.
3. Write one plausible benign explanation and one alternative explanation.

4. Identify at least two additional data sources you would seek in a real, authorised investigation.
5. State whether the evidence supports, weakens, or leaves the hypothesis unresolved.

5. Query-thinking example

The following KQL-shaped example illustrates the logic for a Microsoft Sentinel-style table. Column and table names vary by data source, so adapt it only inside an authorised lab with matching fields.

```
SecurityEvent
| where EventID in (4624, 4625)
| summarize failures=countif(EventID == 4625), successes=countif(EventID == 4624) by Account, IpAddress
| where failures >= 5 and successes > 0
```

IMPORTANT This example finds a pattern worth reviewing; it cannot tell you by itself whether an account was compromised.

6. Finding template

Write down	Prompt
Question	What did I try to find?
Evidence	Which event IDs, times, hosts, users, and fields support the finding?
Alternative	What legitimate activity could explain the pattern?
Confidence	Low / medium / high, with a reason.
Unknowns	What data is missing or incomplete?
Next check	What is the safest, most useful authorised follow-up?

Further learning

- MITRE ATT&CK knowledge base
- Microsoft threat hunting overview
- KQL overview

A real lab for these notes

Topic: Authentication anomalies, baselines, and hypothesis-driven hunting. These pages connect the existing OpenCyberPath notes to specific provider-hosted exercises. The card label is intentionally generic; the exact task name and direct URL are listed here so you can open the right exercise instead of a whole catalogue.

Concept checkpoints

- Start with a falsifiable question, not a conclusion.
- Compare failed logons with success-after-failure, host, user, source IP, and timeframe.
- RFC 5737 test addresses and example.invalid domains in the bundled CSV are fictional training indicators.
- Repeated failures can be caused by typos, stale credentials, service jobs, or malicious activity; gather context.

Windows Threat Detection 1 *[TryHackMe]*
https://tryhackme.com/room/windowsthreatdetection1
Access note: Account required; verify room access and the lab machine entitlement shown by the provider.

T1110-003 *[CyberDefenders]*
https://cyberdefenders.org/blueteam-ctf-challenges/t1110003/
Access note: Direct challenge page about credential-access and threat-hunting patterns. Sign-in, downloadable evidence, and current availability can vary.

ACCESS CHECK Open the exact task page and verify the current login, account tier, VM launch button, free/paid terms, and any cost notes before starting. A provider may change availability or retire a challenge.

Before you click Start

- Write down the provider's target/scope and use only the instance it creates for you.
- Keep this notebook beside the exercise. Use its original course data before trying commands on a local machine.
- Capture timestamps, filenames, row/packet IDs, and a short explanation; redact credentials, tokens, and personal data.
- If a page says retired, premium, or downloadable-only, use the alternative listed in the access note or return to the matching interactive lab above.

Commands you can explain

All bundled rows are synthetic. The commands parse local files only; do not run password attempts or test a live account.

First download the matching sample file(s) from the existing card when a command names one. Keep the data in a practice folder and never execute strings stored in evidence.

POWERSHELL / CHRONOLOGICAL VIEW

```
Import-Csv .\threat-hunt-auth-dns-samples.csv | Sort-Object timestamp_utc | Format-Table timestamp_utc,host,user,source_ip,result,label -AutoSize
```

Read the downloaded synthetic CSV in timestamp order.

POWERSHELL / GROUP FAILURES

```
Import-Csv .\threat-hunt-auth-dns-samples.csv | Where-Object {$_.result -eq 'failure'} | Group-Object host,source_ip | Sort-Object Count -Descending | Select-Object Count,Name
```

Find repeated synthetic failures by host and source without making any network connection.

POWERSHELL / EXAMINE ONE HOST

```
Import-Csv .\threat-hunt-auth-dns-samples.csv | Where-Object {$_.host -eq 'TRAIN-WIN-02'} | Select-Object timestamp_utc,user,source_ip,result,label
```

Inspect a supplied fixture host; the name is fictional.

LINUX / VIEW FAILED ROWS

```
grep -i ",failure," threat-hunt-auth-dns-samples.csv
```

Filter the CSV to rows containing a failed result; confirm the header/column order first.

LINUX / COUNT LABEL COLUMN

```
awk -F, 'NR>1 {count[$8]++} END {for (label in count) print label, count[label]}' threat-hunt-auth-dns-samples.csv
```

Count the label field in this specific nine-column CSV. CSV quoting rules may require Python for more complex files.

COMMAND HABIT Check the current working directory, shell, target, and expected effect before pressing Enter. Save the exact command and the output that mattered.

Guided lab run

- 1. Write your hypothesis in one sentence, including the host, activity type, and time window.
- 2. Load the sample CSV and check its headers, row count, and UTC timestamps.
- 3. Group failures by host and source, then inspect any success immediately after a burst.
- 4. Compare the pattern with benign baseline rows before assigning a label.
- 5. Write what the sample supports, what it cannot establish, and the next telemetry needed.

Evidence checklist

- [ ] What exact lab instance or sample file was used?
- [ ] Which command/query and time range produced the observation?
- [ ] What is the directly observed fact, separate from your inference?
- [ ] What benign/alternative explanation could fit?
- [ ] What extra record or control would help confirm the conclusion?

Check your understanding

Q What do repeated failures followed by a success prove?

Think it through: They justify closer review but do not by themselves prove account compromise.

Q Why use UTC in the evidence timeline?

Think it through: A consistent timezone reduces ordering mistakes when sources come from different systems.

Q What is a useful next data source?

Think it through: MFA outcomes, sign-in risk, endpoint process events, VPN/RDP context, and the user's expected activity.

NOTEBOOK CLOSE-OUT Finish with: Observation / Evidence source / Why it matters / Alternative explanation / Next check / Limitation. A careful "not enough evidence yet" is a valid professional result.